

Datavalidering i Python med Pandera

1. Inledning

Datakvalitet är en viktig del av arbete med Data Science. Analyser, visualiseringar och maskininlärningsmodeller bygger på den data som används. Om datan innehåller felaktiga värden, saknade värden eller värden som inte följer de regler som förväntas kan resultatet av analysen bli felaktigt eller missvisande.

I denna fördjupningsuppgift har jag därför valt att undersöka datavalidering i Python med biblioteket Pandera. Pandera gör det möjligt att definiera regler för hur ett dataset ska se ut och sedan automatiskt kontrollera om datan följer dessa regler.

I projektet har jag byggt en mindre fungerande Pythonlösning där två CSV-filer läses in med Pandas och sedan valideras med Pandera. Den ena filen innehåller korrekt data och den andra innehåller medvetet felaktiga värden för att demonstrera vilka typer av problem Pandera kan upptäcka.

2. Syfte

Syftet med arbetet var att lära mig grunderna i hur datavalidering fungerar i Python och hur biblioteket Pandera kan användas tillsammans med Pandas.

Jag ville framför allt lära mig att:

- skapa ett valideringsschema för ett dataset,
- definiera regler för olika kolumner,
- kontrollera saknade värden,
- kontrollera att numeriska värden ligger inom tillåtna gränser,
- fånga och visa flera valideringsfel vid samma körning,
- förstå varför datavalidering är relevant inom Data Science.

Målet var inte att bygga ett stort system eller en avancerad datapipeline. Fokus låg istället på att skapa en mindre lösning som fungerade och som jag kunde förstå och förklara.

3. Området och dess relevans

Pandera är ett Pythonbibliotek som kan användas för att kontrollera strukturerad data. I mitt projekt används Pandera tillsammans med Pandas DataFrames.

Med Pandera går det att skapa ett schema som beskriver vilka regler ett dataset ska följa. Ett schema kan till exempel beskriva vilka kolumner som ska finnas, vilka datatyper kolumnerna ska innehålla och vilka värden som är tillåtna.

Detta är relevant inom Data Science eftersom data ofta behöver kontrolleras innan den används vidare.

Om en kolumn som ska innehålla ålder exempelvis innehåller värdet `-5` eller `150` kan det påverka en analys. På samma sätt kan saknade värden eller negativa värden där de inte är rimliga skapa problem.

Genom att automatisera dessa kontroller kan fel upptäckas tidigare. Det minskar risken för att felaktig data används i senare delar av ett arbetsflöde.

4. Viktiga begrepp

DataFrame

En DataFrame är en tabellstruktur i Pandas där data organiseras i rader och kolumner. I projektet läses CSV-filerna in som DataFrames innan de valideras.

Exempel:

```
df = pd.read_csv(file_path)
```

Här läser Pandas in en CSV-fil och sparar datan i variabeln `df`.

Schema

Ett schema beskriver vilka regler datan måste följa.

I projektet skapade jag ett schema för kolumnerna:

- `name`
- `age`
- `salary`

Pandera jämför sedan datan i DataFramen med reglerna i schemat.

Check

`Check` används för att skapa regler för värden.

För ålder använder jag:

```
Check.in_range(0, 100)
```

Det innebär att värdet måste ligga mellan 0 och 100.

För lön använder jag:

```
Check.ge(0)
```

`ge` betyder "greater than or equal to". Regeln innebär därför att värdet måste vara större än eller lika med 0.

nullable=False

Jag använder även:

```
nullable=False
```

Det betyder att kolumnen inte får innehålla saknade värden.

lazy=True

När datan valideras använder jag:

```
schema.validate(df, lazy=True)
```

`lazy=True` är en viktig del av lösningen. Det gör att Pandera försöker samla in flera valideringsfel under samma körning istället för att avsluta direkt när det första felet upptäcks.

Detta gör resultatet tydligare eftersom flera problem i datasetet kan visas samtidigt.

5. Genomförande

Projektet byggdes i Python och innehåller två egenkonstruerade CSV-filer.

Den första filen heter:

```
valid_data.csv
```

Den innehåller korrekt data:

```
name,age,salary
Ali,25,30000
Sara,31,35000
Ahmed,22,28000
Fatima,45,42000
Omar,36,40000
```

Den andra filen heter:

invalid_data.csv

Den innehåller medvetet felaktig data:

```
name,age,salary
Ali,25,30000
Sara,-5,35000
Ahmed,150,28000
Fatima,45,
Omar,36,-2000
```

Jag skapade den felaktiga datan för att kunna testa flera typer av fel.

Sara har åldern **-5**, vilket är lägre än den tillåtna gränsen.

Ahmed har åldern **150**, vilket är högre än den tillåtna gränsen.

Fatima saknar ett värde för **salary**.

Omar har lönen **-2000**, vilket är lägre än den tillåtna gränsen.

Valideringsschema

I Python skapade jag sedan ett schema med Pandera.

Förenklat ser reglerna ut så här:

```
schema = DataFrameSchema({
    "name": Column(str, nullable=False),

    "age": Column(
        int,
        Check.in_range(0, 100),
        nullable=False
    ),

    "salary": Column(
        float,
        Check.ge(0),
        nullable=False,
        coerce=True
    )
})
```

För **name** krävs text och värdet får inte saknas.

För **age** krävs heltal och värdet måste ligga mellan 0 och 100.

För **salary** kontrolleras att värdet är större än eller lika med 0 och att värdet inte saknas.

Validering av datan

Programmet läser först in CSV-filen med Pandas:

```
df = pd.read_csv(file_path)
```

Därefter körs Pandera-valideringen:

```
schema.validate(df, lazy=True)
```

Om datan följer reglerna skriver programmet ut:

Data är godkänd!

Om datan inte följer reglerna fångas valideringsfelen och skrivs ut i terminalen.

Jag använder `try` och `except` för att hantera detta. Det gör att programmet inte bara kraschar vid ett valideringsfel utan istället kan visa vilka problem som Pandera har hittat.

Under arbetet fick jag även en varning från Pandera eftersom vissa importter hade ändrats i en nyare version av biblioteket. Jag ändrade därför importerna så att de Pandas-specifika funktionerna importerades från `pandera.pandas`. Efter ändringen försvann varningen och programmet fungerade som tänkt.

6. Resultat

När programmet kördes med `valid_data.csv` blev resultatet:

Data är godkänd!

Det betyder att samtliga värden i filen följde de regler som definierades i schemat.

När programmet kördes med `invalid_data.csv` identifierade Pandera fyra problem:

column	failure_case	check
age	-5.0	in_range(0, 100)
age	150.0	in_range(0, 100)
salary	NaN	not_nullable
salary	-2000.0	greater_than_or_equal_to(0)

Resultatet visar att programmet kunde upptäcka:

- en negativ ålder,
- en ålder över den tillåtna gränsen,
- ett saknat värde,
- en negativ lön.

Det visar även varför `lazy=True` var användbart. Istället för att endast visa det första problemet kunde Pandera samla alla fyra valideringsfelen.

Lösningen uppfyllde därför målet att demonstrera hur Pandera kan användas för att automatiskt hitta flera typer av problem i data.

7. Begränsningar och möjliga förbättringar

Projektet är medvetet avgränsat.

Jag använder ett litet egenkonstruerat dataset istället för ett stort verkligt dataset. Det gjorde det enklare att kontrollera vilka fel som skulle finnas och visa tydligt hur Pandera reagerade.

Lösningen kontrollerar endast några grundläggande regler. Den kontrollerar exempelvis inte datum, e-postadresser, unika ID:n eller mer avancerade beroenden mellan olika kolumner.

Programmet använder inte heller en databas, ett API eller någon större datapipeline.

Om projektet skulle utvecklas vidare skulle jag kunna:

- använda ett större verkligt dataset,
- lägga till fler kolumner,
- skapa fler valideringsregler,
- kontrollera kategoriska värden,
- kontrollera datum,
- kontrollera unika värden,
- spara valideringsfelen till en separat fil,
- använda valideringen som en del av ett större dataflöde.

8. Koppling till yrkesrollen

Datavalidering är relevant för en Data Scientist och dataanalytiker eftersom kvaliteten på datan påverkar resultatet av arbetet.

Innan data används för exempelvis analys, visualisering eller maskininlärning behöver man kunna kontrollera att den är rimlig och användbar.

Pandera kan exempelvis användas för att automatiskt kontrollera inkommande data innan den används vidare.

Om reglerna inte uppfylls kan problemet upptäckas tidigt istället för att felaktig data påverkar analysen längre fram.

Det gör datavalidering till en viktig del av arbetet med att skapa tillförlitliga dataflöden och analyser.

9. Självreflexion

9.1 Vad lärde jag mig som jag inte kunde innan?

Jag lärde mig hur Pandera kan användas tillsammans med Pandas för att kontrollera att data följer bestämda regler.

Jag lärde mig också hur man skapar ett valideringsschema och hur `Check` kan användas för att kontrollera olika värden.

En viktig sak jag lärde mig var hur `lazy=True` fungerar och varför det är användbart när man vill upptäcka flera fel under samma körning.

9.2 Vad var svårast att förstå eller genomföra?

Det svåraste i början var att förstå hur Pandera-schemat skulle byggas upp och vad de olika delarna betydde.

Jag behövde förstå skillnaden mellan exempelvis:

```
nullable=False
```

```
Check.in_range()
```

och

```
Check.ge().
```

Jag fick även en varning kring hur Pandera skulle importeras. Genom att ändra importen till `pandera.pandas` kunde jag lösa problemet.

9.3 Vilket tekniskt val är jag mest nöjd med och varför?

Jag är mest nöjd med att använda `lazy=True`.

Det gjorde att programmet kunde hitta flera fel samtidigt istället för att avsluta efter det första valideringsfelet.

Det var särskilt användbart i mitt projekt eftersom jag medvetet hade lagt in flera olika problem i testdatan.

Resultatet blev därför tydligare och visade bättre vad Pandera kan användas till.

9.4 Vad hade jag gjort annorlunda om jag började om?

Om jag började om hade jag först planerat alla regler och alla testfall innan jag började skriva själva programmet.

Då hade jag redan från början haft en tydlig lista över vilka fel jag ville testa och vilka Pandera-funktioner jag behövde använda.

Det hade gjort arbetsprocessen mer strukturerad.

9.5 Vad skulle vara ett naturligt nästa steg om jag fortsatte arbetet?

Ett naturligt nästa steg skulle vara att använda ett större och mer realistiskt dataset.

Jag skulle även kunna lägga till fler valideringsregler, exempelvis för datum, kategorier och unika ID:n.

En annan utveckling skulle kunna vara att automatiskt skapa en rapport med alla fel som hittas.

9.6 Vilket betyg tycker jag själv att arbetet motsvarar?

Jag tycker att arbetet motsvarar **Godkänt**.

Jag har genomfört en tydligt avgränsad fördjupning inom Python, använt dokumentation för att sätta mig in i Pandera och byggt en fungerande lösning.

Programmet kan läsa in data, kontrollera flera olika regler och visa både korrekt data och flera typer av felaktig data.

Jag kan även förklara de viktigaste delarna av lösningen och varför datavalidering är relevant inom Data Science.

10. Slutsats

Syftet med projektet var att lära mig grunderna i datavalidering med Pandera och bygga en mindre fungerande lösning.

Resultatet visar att Pandera kan användas för att automatiskt kontrollera om ett dataset följer bestämda regler.

I projektet kunde Pandera identifiera negativ ålder, för hög ålder, ett saknat värde och negativ lön.

Jag har genom arbetet fått en bättre förståelse för hur ett valideringsschema fungerar och varför datavalidering är viktigt inom Data Science.

Projektet visar också att även en relativt enkel valideringslösning kan hjälpa till att upptäcka problem innan data används vidare i en analys eller modell.

11. Källor

Pandera. *Pandera Documentation*.
<https://pandera.readthedocs.io/>

Pandas. *Pandas Documentation*.
<https://pandas.pydata.org/docs/>